Bulkhead Pattern

Category: Resilience **One-liner:** Partition a system into isolated compartments with their own dedicated resources, so a failure or overload in one cannot consume the resources everything else needs.

Part 0 — Concepts you need first

What is a thread pool?

A fixed collection of worker **threads** kept ready: work arrives, a free thread takes it, then returns to the pool (introduced in Circuit Breaker Part 0). Pools are fixed-size because threads are expensive — each needs ~1 MB of **stack** (memory for its method calls and locals), and with thousands the **operating system** spends more time switching between them than working.

The consequence that drives this chapter: the pool is a *shared, finite resource*. If something exhausts it, **every** piece of work in that program stops — not just the work that caused the problem.

What does "resource exhaustion" mean?

Using up all of a limited resource so nothing else gets any:

- **Threads** — all busy, so no new work can start
- **Memory** — allocated, so new allocations fail
- **Database connections** — **connection pools** (reused open connections) are also fixed-size and exhaustible
- **Network bandwidth / file handles / CPU** — a **file handle** is the OS's ticket for one open file or connection, and there's a limit

What is a "noisy neighbour"?

A component consuming disproportionate shared resources and degrading everything sharing them. The term comes from cloud hosting (one tenant's heavy usage slowing others on the same machine); in your app, one slow **dependency** can be a noisy neighbour to every other feature.

Why does one slow dependency block unrelated work?

The crux. Your service has **one pool of 200 threads** and calls Inventory (usually busy) and Shipping (usually quiet). Inventory slows to 30 s per call instead of 50 ms:

1. Requests needing Inventory each take a thread
2. Each blocks for 30 s
3. At any moderate rate, all 200 threads end up blocked on Inventory
4. A request needing only Shipping arrives — **no thread is available**

Shipping is perfectly healthy. Its requests still fail, because Inventory ate all the threads. That's what this pattern prevents. (It's the same thread-starvation mechanism behind cascading failure.)

Part 1 — Where the name comes from

A ship's hull is divided into **watertight compartments** — bulkheads. A breach floods **one compartment**, the ship stays afloat, and there's time for rescue. Without them one breach floods everything: a **cascading failure in a physical system**.

In **distributed systems**, a bulkhead **isolates parts of the system so that if one part fails, it doesn't cause the rest to fail.**

Part 2 — The problem: failure propagation

Services fail — bugs, resource exhaustion, network issues — and a failure **ripples** to everything depending on it.

The e-commerce walkthrough

A **microservices** e-commerce platform with user management, inventory, payments:

1. **Inventory** fails due to a database **outage**
2. **User service**, which shows product availability, **also starts failing**
3. **Payment service**, which checks inventory before charging, **likewise fails**
4. Soon **the entire platform is unavailable** — from one service's failure. A classic cascading failure.

Part 3 — The solution: isolation

The essence is **isolation**: partition components into units that operate **independently**, so if one fails the others continue. Bulkheads are **failure containment** — they limit the **scope** of a failure, so unaffected parts keep serving and **availability** is preserved as far as possible.

Applied: with inventory in its own bulkhead, user and payment services keep offering limited functionality — browsing products, viewing past orders — while inventory is down.

The half people miss

By stopping the inventory failure from **consuming system-wide resources** (CPU, memory), the pattern keeps those resources **available to the remaining services**.

> Isolating *resources* matters as much as isolating *code*. Two services can be perfectly separate in the **source tree** and still kill each other through a shared thread pool. The bulkhead is about the resources, not the modules.

Not a magic solution

A ship with a big enough breach may sink anyway; a total network outage may still take a system down. Bulkheads **buy time and maintain some level of service** — significant mitigation, not prevention.

Part 4 — Architecture

The architecture revolves around **isolation** and **resources**.

What is a bulkhead, concretely?

A **partition**: a separate **service**, a group of related **processes**, a set of **threads** (a thread pool — the most common implementation), or a dedicated **CPU or memory area**. The requirement: each operates **independently**.

What are the resources?

Anything the bulkheads need — CPU time, memory, bandwidth, database connections. Each bulkhead gets a **dedicated share**, so one failure can't consume everything.

Step 1 — Divide the system into bulkheads

Isolate services **prone to failure**, services doing **resource-intensive tasks**, or group by **function** or by **type of users served**. In e-commerce: separate bulkheads for user management, inventory, payments. Aim: if one fails **or becomes a resource hog**, the others aren't directly affected.

Step 2 — Allocate resources

Proportional to needs and importance: more for **payment processing** (financial transactions) than **user management** (profiles), based on expected load, performance requirements, and the consequences of failure. This is the critical mechanism: a failing bulkhead that over-consumes still **can't take the other bulkheads' allocations**.

Step 3 — Manage interactions between bulkheads

- **Keep interactions to a minimum** — the more one bulkhead depends on others, the likelier failure spreads.
- **When necessary, isolate them** — e.g. via a **message queue** or an **API gateway**, so an inventory failure doesn't directly hit user management.

> Why a queue helps: with a direct call the caller's thread waits while the callee struggles. With a queue the caller drops a message and moves on; the failure becomes "messages accumulate" instead of "caller's threads are all blocked". (See Event-Driven Architecture.)

Part 5 — Inner workings

Isolation in practice

Each bulkhead might run in its **own process or container**, have its **own database and cache**, and use its **own thread pool**. But:

> Isolation is **not just technical** — it needs **organisational commitment**. Changes to one bulkhead shouldn't require changes to others; interdependencies must be minimised and managed.

Perfect technical isolation can be destroyed in six months by teams adding convenient direct dependencies. It must be defended, not just built.

Resource allocation — fairness and efficiency

Aim for **fairness** (each bulkhead gets what it needs) and **efficiency** (nothing wasted): **resource quotas or limits** per bulkhead, fair scheduling, and **dynamic adjustment** — e.g. temporarily lending an idle bulkhead's resources to one under a surge.

Failure detection and mitigation

The system must **detect, isolate and mitigate** a bulkhead failure: **continuous monitoring** of each bulkhead's health; **alerts** plus **automatic recovery**; **throttling** (deliberately slowing) or **restarting** a bulkhead that over-consumes; **rerouting traffic** or **bringing up new instances** if it stays unresponsive.

It's not a silver bullet

It can't prevent failures or guarantee 100% **uptime**. Complement it with **redundancy, replication, load balancing, and caching** (§ Fault tolerance).

Part 6 — Worked example: thread-pool bulkheads in Java

The scenario

An **Order Service** calls **Inventory** (often high traffic) and **Shipping** (usually less load). **Without bulkheads**, an Inventory spike can clog Order Service and stop it completing Shipping work.

Creating separate thread pools

Java's `ExecutorService` (the standard interface for running tasks on a thread pool):

```
ExecutorService inventoryExecutor = Executors.newFixedThreadPool(100);
ExecutorService shippingExecutor  = Executors.newFixedThreadPool(50);
```

`newFixedThreadPool(100)` = exactly 100 threads; extra tasks queue until one frees. Sizes follow expected load — Inventory gets 100, Shipping 50. **These two pools are the bulkheads**: threads in one can't be consumed by work for the other.

Using them

```
public void createOrder(Order order) {
    inventoryExecutor.submit(() -> {
        inventoryService.checkAndUpdateInventory(order);
    });

    shippingExecutor.submit(() -> {
        shippingService.scheduleShipping(order);
    });
}
```

The bulkhead in action

1. Requests start slowing the **Inventory Service**
2. Threads in `inventoryExecutor` get stuck — potentially all 100
3. **The 50 threads in `shippingExecutor` are unaffected** — separate pool
4. `createOrder` **keeps scheduling shipping** while inventory checks are delayed

Versus the single-pool version: all 200 threads blocked on Inventory and shipping failing for no reason of its own. **The only change is drawing from two buckets instead of one.**

What the example deliberately leaves out

- **Error handling** — no response, or an **exception**?
- **Timeouts** — a stuck thread needs a bound *(pair with Circuit Breaker)*
- **Pool sizing** — how to choose the numbers (Part 14D)
- `submit()` returns a `Future` (holder of the task's result or error) that's ignored here, so **failures are silently swallowed** (pitfall 3).

Part 7 — Performance implications

Benefits

- **Efficient resource utilization** — work continues while one service is sluggish; no whole-system freeze from one faulty component.
- **Load balancing** — separate pools distribute load, tunable per task or service.
- **Response time** — a delay in one part doesn't hold up others, which are served swiftly.

Considerations

Determining bulkhead sizes — a real challenge, often **trial and error**. Starting point: size each pool to the **concurrency** the dependency can actually handle. If Inventory degrades past 100 concurrent requests, a larger pool just queues work inside *it* instead of inside you — worse, because you lose visibility.

Monitoring and adjusting — as load evolves, revisit sizes; tools like **VisualVM** (shows a running JVM's threads, memory, CPU) reveal thread usage and **bottlenecks**.

Error handling — requests can still fail or stall when a service is down; handle with a **timeout** or a **Circuit Breaker**.

Scalability concerns:

> The Bulkhead pattern improves **resilience** but **does not directly address scalability**. Under significant load growth, creating more threads causes **excessive context switching** (the CPU repeatedly saving one thread's state and loading another's — pure overhead) and *decreases* performance. For scale, use horizontal scaling or **asynchronous** processing.

⚠️ **The point beginners get wrong most often.** Bulkheads bound damage; they don't add capacity. Bigger pools as a load fix make things slower.

Part 8 — Use cases

- **E-commerce** — search, cart and ordering are interconnected but independent. An overwhelmed **search service** (traffic spike or bug) would otherwise slow the whole platform — **even cart and ordering**.
- **Online gaming** — each game a separate service; **player load in one game shouldn't affect others**.
- **Financial systems** — account management, transaction processing and reporting can function independently.

> **A delay generating reports shouldn't slow transaction processing** — the cleanest illustration: reporting is heavy and slow, transactions are **latency**-critical, and there's no reason the former should interfere with the latter.

Part 9 — System design example: music streaming (Spotify-like)

Services: streaming, user management, playlist management, recommendations.

Bulkheading services (coarse-grained)

- **User management isolated from streaming** — a surge of new sign-ups doesn't affect playback; **users keep listening**.
- **Playlist management isolated from recommendations** — slow, analysis-heavy recommendations don't stop users managing playlists.

Bulkheading at a lower level (fine-grained)

Within one service: put **fetching a song** (from the song database) and **streaming the song** to the user in **separate thread pools**, so a slow song database **doesn't interrupt streaming of songs already fetched**.

> The most useful takeaway: bulkheads apply *between* services **and** *within* one. The fine-grained version often saves the experience — music that keeps playing while search is broken.

Part 10 — Bulkhead vs. Circuit Breaker

Complementary, not alternatives:

	Bulkhead	Circuit Breaker
Mechanism	Partitions resources	Stops calls after repeated failures
Protects against	Resource exhaustion spreading	Wasted calls to a known-dead service
When it acts	Always — it's structural	On detecting failures — it's reactive
Analogy	Watertight compartments	A fuse that trips
Limits	*How much* damage one failure can do	*How long* you keep trying

Use both. The bulkhead caps the **blast radius** (how much of the system one failure can damage); the breaker stops you feeding the fire. Inventory can only ever consume its own 100 threads, and a breaker on Inventory stops those threads being wasted once it's clearly down.

Part 11 — Common beginner misconceptions

- **"Bulkhead means microservices."** You can bulkhead inside a **monolith** by giving different **workloads** different pools — often the highest-value place to start.
- **"Bigger pools are safer."** Past a point the opposite: threads cost memory and context switches. Match the pool to what the dependency can handle.
- **"It's the same as rate limiting." Rate limiting** caps how many requests you *accept* per time window; bulkheading caps how many resources a category of work can *consume*.
- **"Isolation is a one-time build."** It erodes as teams add cross-dependencies; it needs ongoing defence — an organisational commitment.
- **"Bulkheads help with load."** They help with *failure*. For load, scale horizontally.

Part 12 — Check your understanding

1. Why can a slow Inventory Service cause Shipping requests to fail, when Shipping is healthy?
2. What are the two things a bulkhead isolates, and which do people forget?
3. Why does a message queue between bulkheads help contain failure?
4. Why doesn't the Bulkhead pattern help with scalability?
5. Give an example of fine-grained bulkheading inside a single service.
6. How does Bulkhead differ from Circuit Breaker?

(Answers: 1 — they share one thread pool, and Inventory's blocked threads consume all of it; 2 — code and resources, and people forget resources; 3 — the caller drops a message and moves on instead of holding a thread waiting for a struggling callee; 4 — more threads cause excessive context switching and degrade performance, so capacity comes from horizontal scaling; 5 — separate thread pools for fetching songs versus streaming already-fetched songs; 6 — bulkhead limits how much damage a failure can do and is always active, breaker limits how long you keep trying and activates on detected failures.)

Part 13 — Interview soundbite (30 seconds)

> "Bulkhead comes from ship design — watertight compartments so one breach doesn't sink the ship. In software you partition resources so a failing dependency can't consume everything. Classic case: an Order Service calls Inventory and Shipping from one shared thread pool; Inventory gets slow, every thread ends up blocked on it, and Shipping calls fail though Shipping is healthy. The fix is separate pools sized per dependency, so Inventory's problems are bounded to its own hundred threads. It applies between services or inside one — e.g. separating song fetching from streaming so a slow database doesn't interrupt playback. It pairs with circuit breakers: bulkhead caps how

> much damage a failure can do, the breaker caps how long you keep trying. Gotchas: pool sizing is trial and error and needs monitoring — start from Little's Law — and it improves resilience, not scalability: more threads for more load just means context switching, so capacity comes from horizontal scaling."

Part 14 — Implementation

A. Runnable demo — see thread starvation, then see it fixed

(**Thread starvation**: work can't run because every thread is taken.)

Full source: `demos/chapters/07-bulkhead/BulkheadDemo.java`

```
cd demos && ./gradlew runCh07
```

The program runs the *same* workload twice. Inventory is slow (`Thread.sleep(2000)`), Shipping fast (50 ms) and healthy; 20 inventory tasks are submitted, then 5 shipping tasks, and it counts shipping completions (with a thread-safe `AtomicInteger`) within 3 s. The only difference between the runs is where the tasks go:

```
// WITHOUT bulkheads: one shared pool of 10
ExecutorService shared = Executors.newFixedThreadPool(POOL);        // POOL = 10
shared.submit(() -> { slowInventory(); return null; });              // x20
shared.submit(() -> { fastShipping(); shippingDone.incrementAndGet(); return null; });  // x5

// WITH bulkheads: separate pools
ExecutorService inventoryPool = Executors.newFixedThreadPool(POOL);
ExecutorService shippingPool  = Executors.newFixedThreadPool(5);
```

B. What you should see

```
=== WITHOUT bulkheads (shared pool of 10) ===
  shipping completed in 3s: 0/5   (finished=false, 3001ms)

=== WITH bulkheads (separate pools) ===
  shipping completed in 3s: 5/5   (3000ms)
```

The whole pattern in two lines. Same workload, same total resources. In the first run twenty slow inventory tasks occupy all ten threads, and five trivial shipping tasks get **zero**. In the second, shipping has its own pool and finishes immediately — while inventory is just as slow. Nothing was made faster; something was made *isolated*.

C. Production — Resilience4j Bulkhead

(Library setup — Maven dependency, Spring AOP — is in Circuit Breaker 16C.) Two flavours, and choosing correctly matters:

Type	Mechanism	Use when
`SemaphoreBulkhead`	Caps **concurrent calls** on the caller's own thread (a **semaphore** is a counter of permits)	Default choice — cheap, no extra threads
`ThreadPoolBulkhead`	Runs calls on a **separate thread pool**	You need true thread isolation, or async execution

```yaml
resilience4j:
  bulkhead:                          # semaphore-based
    instances:
      inventoryService:
        maxConcurrentCalls: 25       # at most 25 in flight
        maxWaitDuration: 100ms       # then reject rather than queue forever
      shippingService:
        maxConcurrentCalls: 10
        maxWaitDuration: 100ms

  thread-pool-bulkhead:              # thread-pool-based
    instances:
      reportGeneration:
        maxThreadPoolSize: 4
        coreThreadPoolSize: 2
        queueCapacity: 20
```

Usage (to **shed load** = reject excess work quickly instead of queuing it; `sku` = stock-keeping unit, a product ID):

```java
@Bulkhead(name = "inventoryService", fallbackMethod = "inventoryFallback")
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}

private Inventory inventoryFallback(String sku, BulkheadFullException e) {
    // Bulkhead full — shed load rather than queue
    return Inventory.unknown(sku);
}
```

Kubernetes-level bulkheads — the same idea for whole services. A **pod**'s `requests` is what it's guaranteed, `limits` the most it may use (`500m` = half a CPU core; `Mi`/`Gi` = mebibytes/gibibytes):

```yaml
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { cpu: "1000m", memory: "1Gi" }   # this pod cannot starve its neighbours
```

Database connection pools are bulkheads too — often the most valuable one (**HikariCP** is Spring Boot's default pool):

```yaml
spring.datasource.hikari:
  maximum-pool-size: 20
  connection-timeout: 3000     # fail fast when the pool is exhausted
```

D. How to size the pools

Start from **Little's Law** (items in progress = arrival rate × time each spends — § Little's law):

```
concurrency = throughput × latency
```

A dependency serving **50 requests/second** at **200ms** average latency has `50 × 0.2 = 10` calls in flight at **steady state**. Size the bulkhead a bit above — say 15 — to absorb bursts.

Key insight: never size it larger than the dependency can actually handle. If Inventory degrades beyond 100 concurrent requests, a bulkhead of 500 just moves the queue from your process into theirs, where you can't see or shed it. Then measure and adjust; a short `maxWaitDuration` (~100ms) means you **shed load** rather than accumulate it.

E. Implementation pitfalls

1. Pools sized by guesswork. Start from Little's Law, then tune with real metrics. "200 because that's the default" protects nothing.

2. Unbounded queues. `Executors.newFixedThreadPool` uses an **unbounded** `LinkedBlockingQueue`; under sustained overload it accumulates tasks until `OutOfMemoryError` — you traded thread exhaustion for memory

exhaustion. Use a bounded queue and an explicit **rejection policy**:

```
new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(50),                    // bounded
    new ThreadPoolExecutor.AbortPolicy());           // reject when full
```

3. Ignoring the `Future`. `submit()` swallows exceptions into a `Future` nobody reads. Use `execute()` with an `UncaughtExceptionHandler` (called when a thread dies from an unhandled error), or check the result.

4. Forgetting the database connection pool. Teams bulkhead HTTP calls and leave one shared 20-connection Hikari pool where one slow query starves every **endpoint**. The connection pool is usually the *scarcest* resource you own.

5. Bulkheading without timeouts. A bounded pool of threads blocked forever is still dead. The bulkhead limits *how many* can be stuck; only a timeout limits *how long*.

6. Too many small pools. ~1 MB of stack per idle thread: twenty pools of fifty threads ≈ a thousand threads and ~1 GB before serving a request. Bulkhead by *dependency*, not endpoint.

7. Treating it as a scaling tool. Repeated because it's the most common error (Part 7): past the CPU's capacity, more threads mean *less* throughput.

Related patterns

- **Circuit Breaker** — the natural companion; stop calling what has already failed
- **Retry** — retries inside a bounded bulkhead can't exhaust the whole system
- **Sidecar** — service meshes can enforce bulkheads at the infrastructure level
- **Event-Driven Architecture** — message queues between components are a natural isolation boundary